Index

Symbols

- @AllArgsConstructor, Defining the domain class, Populating the database using the application's repository, Updating PlaneFinder
- @Autowired, The utility, aka "Springing" into action, Broadcasting aircraft positions to WebSocket connections
- @Bean, Potential Third-Party Option, Converting from Template to Repository
- @BeforeEach, And Now, the Testing
- @Component, @RestController in a Nutshell, A Bit of Polishing, Bringing it all together, Populating the database using the application's repository, Polling PlaneFinder, Polling PlaneFinder, But First, Refactoring
- @Configuration, Potential Third-Party Option-Potential Third-Party Option, Supplying aircraft positions, Consuming aircraft positions, Adding Authentication, Refactoring Aircraft Positions for authentication and authorization, Refactoring PlaneFinder for resource authorization, But First, Refactoring
- @ConfigurationProperties, @ConfigurationProperties-@ConfigurationProperties, Potential Third-Party Option, Autoconfiguration Report
- @Controller, @RestController in a Nutshell, Working with Model and Controller-Working with Model and Controller, Migrating PlaneFinder to RSocket, Migrating Aircraft Positions to RSocket
- @Data, Defining the domain class, Defining the domain class
- @Document, Defining the domain class, Updating the Aircraft Positions application
- @EnableGlobalAuthentication, Authorization
- @EnableWebSecurity, Authorization, Refactoring Aircraft Positions for authentication and authorization
- @Entity, The @Entity-The @Entity, Defining the domain class, Loading Data, Creating and populating with scripts
- @GeneratedValue, Defining the domain class
- @GetMapping, Let's GET Busy, Potential Third-Party Option, Working with Model and Controller

- @Id, Defining the domain class, Defining the domain class, Updating the Aircraft Positions application
- @JsonIgnoreProperties, Defining the domain class, Defining the domain class
- @JsonProperty, Defining the domain class, Defining the domain class
- @MessaageMapping, Migrating PlaneFinder to RSocket
- @MockBean, And Now, the Testing
- @NoArgsConstructor, Defining the domain class
- @Node, Defining the domain class
- @NonNull, Polling PlaneFinder
- @PathVariable, Let's GET Busy, DELETE-ing
- @PostConstruct, A Bit of Polishing
- @RequestMapping, Let's GET Busy-And More, Working with Model and Controller
- @RequestParam, But First, Refactoring
- @RequiredArgsConstructor, Polling PlaneFinder
- @ResponseBody, @RestController in a Nutshell, Migrating Aircraft Positions to RSocket, But First, Refactoring
- @RestController, @RestController in a Nutshell, @Value, Potential Third-Party Option, Working with Model and Controller, Migrating PlaneFinder to RSocket, Migrating Aircraft Positions to RSocket
- @Service, Working with Model and Controller, But First, Refactoring
- @SpringBootApplication, Potential Third-Party Option, Supplying aircraft positions, Adding Authentication
- @Value, @Value-@Value, @ConfigurationProperties, @ConfigurationProperties
- @WebFluxTest, And Now, the Testing

A

- Actuator (see Spring Boot Actuator)
- AdoptOpenJDK, The Spring Initializr
- Aeron, What Is RSocket?
- Aircraft Positions, Apache Maven
  - (see also PlaneFinder)
  - container image from command line, Creating a Container Image from the Command Line-Running the Containerized Application
  - developing Spring MVC, Developing the Aircraft Positions Application-A refreshing flourish
  - forms-based authentication, Adding Spring Security Dependencies-Adding Authentication
  - forms-based authorization, Authorization-Authorization

    - migrating to RSocket, Migrating Aircraft Positions to RSocket-Migrating Aircraft Positions to RSocket
    - OpenID Connect and OAuth2, Aircraft Positions Client Application-Refactoring PlaneFinder for resource authorization
    - pom.xml, Apache Maven
    - testing reactive applications, Testing Reactive Applications-Testing Aircraft Positions search capabilities
    - WebSocket, Creating Conversations with WebSocket-The results
- Amazon Kinesis, Supplying aircraft positions
- Apache Kafka, Passing Messages, Supplying aircraft positions
- Apache Maven, Apache Maven-Apache Maven, Choosing Between Maven and Gradle, The Spring Initializr-The Spring Initializr, Staying In Integrated Development Environments (IDEs), Updating the Aircraft Positions application-Updating the Aircraft Positions application
- Apache NetBeans, Staying In Integrated Development Environments (IDEs)
- Apache RocketMQ, Supplying aircraft positions
- API (Application Programming Interface), Creating Your First Spring Boot REST API
    - (see also REST API)
- API- or database-specific scripts, API- or database-specific scripts-Creating and populating with scripts
- ApplicationContext, Refactoring Aircraft Positions for authentication and authorization
- asynchronous processing, Reactive Streams, Introduction to Reactive Programming-Introduction to Reactive Programming, What Is RSocket?
- Atom, Staying In Integrated Development Environments (IDEs)
- authentication and authorization, Securing Your Spring Boot Application-Authorization, Authorization
    - (see also Spring Security)
- authentication(), Adding Authentication
- Authorization Code Flow with PKCE, Refactoring PlaneFinder for resource authorization
- autoconfiguration, Autoconfiguration-Autoconfiguration, Priming Autoconfig for Database Access, Autoconfiguration Report-Autoconfiguration Report
- Azure Event Hubs, Supplying aircraft positions

B

- bean creation methods, Spring Security, Adding Authentication
- bearer tokens (see JWT)
- Bills of Materials (see BOMs)

- BlockHound, ReactorDebugAgent.init()
- blocking versus non-blocking code, Project Reactor, Updating PlaneFinder, When Reactive?
- blue-green deployment, Revisiting the Spring Boot Executable JAR
- BOMs (Bills of Materials), Starters for Simplified Dependency Management, Required dependencies
- builder(), Adding Authentication

C

- Cache-Control response header, Adding Spring Security Dependencies
- checkpoint(), Checkpoints
- checkpoints, Hooks.onOperatorDebug(), Checkpoints-Checkpoints
- CLI (Command Line Interface), The Spring Initializr, Straight Outta Commandline-Straight Outta Commandline
- clickjacking, Adding Spring Security Dependencies
- ClientRegistrationRepository, Refactoring Aircraft Positions for authentication and authorization, Refactoring Aircraft Positions for authentication and authorization
- Cloud Native (Paketo) Buildpacks, Deploying Spring Boot Applications to Containers-Dive
- cold publishers, Updating PlaneFinder
- Command Line Interface (see CLI)
- command line, container images from, Creating a Container Image from the Command Line-Running the Containerized Application
- CommandLineRunner, Updating PlaneFinder, Updating PlaneFinder
- communications (see Spring MVC)
- configuration, Configuring and Inspecting Your Spring Boot App-Turning Up the Volume on Logging with Actuator
  - @Value, @Value-@Value, @ConfigurationProperties, @ConfigurationProperties
  - Actuator, Actuator-Turning Up the Volume on Logging with Actuator
  - autoconfiguration, Autoconfiguration-Autoconfiguration, Priming Autoconfig for Database Access, Autoconfiguration Report-Autoconfiguration Report
  - R2DBC with H2, Updating PlaneFinder
  - Spring Boot plug-in for, Deploying Spring Boot Applications to Containers
- configure(HttpSecurity http), Authorization
- ConnectionFactoryInitializer, Updating PlaneFinder
- Consumer, RabbitMQ, Consuming aircraft positions, Application properties
- container images, Deploying Spring Boot Applications to Containers-Dive

    - from command line, Creating a Container Image from the Command Line-Running the Containerized Application
    - from IDE, Creating a Container Image from an IDE-Creating a Container Image from an IDE
    - running containerized application, Running the Containerized Application
    - utilities for examining, Utilities for Examining Spring Boot Application Container Images-Dive
    - verifying the image, Verifying the Image Exists
- Cross Site Scripting (see XSS)
- CRUD, The Repository
- CrudRepository, The Repository-The utility, aka “Springing” into action, Converting from Template to Repository, Creating the repository interface, Creating the repository interface, Creating the repository interface, Creating the repository interface

D

- Dark UI toggle, The Spring Initializr
- Data Definition Language (see DDL)
- data keyword, Kotlin, Defining the domain class
- Data Manipulation Language (see DML)
- data storage and retrieval (see Spring Data)
- database access, Adding Database Access to Your Spring Boot App-A Bit of Polishing
    - adding a dependency, Adding a Database Dependency-Adding a Database Dependency
    - adding code, Adding Code-The utility, aka “Springing” into action
    - autoconfig priming, Priming Autoconfig for Database Access
    - polishing the code, A Bit of Polishing-A Bit of Polishing
    - saving and retrieving data, Saving and Retrieving Data-Saving and Retrieving Data
- DataLoader, A Bit of Polishing, Populating the database using the application’s repository
- DbConxInit, Updating PlaneFinder
- DDL (Data Definition Language), Loading Data, Creating and populating with scripts, Updating PlaneFinder
- Debugger for Java, Staying In Integrated Development Environments (IDEs)
- debugging, Configuring and Inspecting Your Spring Boot App
- declarative approach with Reactive Streams, Updating PlaneFinder
- DELETE, DELETE-ing, Trust, but Verify., Saving and Retrieving Data

- deleteById(), The utility, aka “Springing” into action
- dependency management
  - database access, Adding a Database Dependency-Adding a Database Dependency
  - extracting dependencies, Exploding JARs-Exploding JARs
  - simplified, Starters for Simplified Dependency Management-Starters for Simplified Dependency Management
- deployment of Spring Boot applications, Deploying Your Spring Boot Application-Dive
  - container images, Deploying Spring Boot Applications to Containers-Dive
  - executable JARs, Revisiting the Spring Boot Executable JAR-Exploding JARs
- diagnosing and debugging
  - Hooks.onOperatorDebug(), Hooks.onOperatorDebug()-Checkpoints
  - ReactorDebugAgent.init(), ReactorDebugAgent.init()-ReactorDebugAgent.init()
- dive utility, Dive
- DML (Data Manipulation Language), Loading Data, Creating and populating with scripts
- Docker, Contacting the Controller, Deploying Spring Boot Applications to Containers, Creating a Container Image from an IDE
- Docker Hub, Verifying the Image Exists
- domain classes, Creating a Simple Domain-Creating a Simple Domain, Defining Entities
- Domain-Driven Design (Evans), Defining Entities
- DSL (Domain Specific Language), Gradle, Initializing the Project, Initializing the Project

E

- Early Access (EA), Choosing Between Maven and Gradle
- Eclipse, Staying In Integrated Development Environments (IDEs)
- encoded password format, Adding Authentication
- encoder, password, Adding Authentication
- encryption versus encoding, Adding Authentication
- enviromental awareness (Actuator), Becoming More Environmentally Aware Using Actuator-Becoming More Environmentally Aware Using Actuator
- Evans, Eric, Defining Entities
- executable JAR files, Executable JARs for Simplified Deployment-Executable JARs for Simplified Deployment, Revisiting the Spring Boot Executable JAR-Building a “Fully Executable” Spring Boot JAR

  - building, Building a “Fully Executable” Spring Boot JAR-Building a “Fully Executable” Spring Boot JAR
  - deployment to containers, Deploying Spring Boot Applications to Containers-Running the Containerized Application
  - extracting (rehydrating) dependencies, Exploding JARs-Exploding JARs

F

- fan-out pattern, messaging, Application properties
- Fielding, Roy, The Hows and Whys of APIs
- filter chains, Spring Security, Security Filter Chains
- findAll(), The utility, aka “Springing” into action
- findById(), The utility, aka “Springing” into action
- Flux, Bringing it all together, Project Reactor, Updating PlaneFinder, Testing Reactive Applications, But First, Refactoring
- FlywayDB, Other mechanisms
- FreeMarker, End User Interactions Using Template Engines

G

- GET, GET-ting-Let’s GET Busy, Trust, but Verify, Saving and Retrieving Data
- Goodman, Alex, Dive
- Google Pub/Sub, Supplying aircraft positions
- Gradle, Gradle-Choosing Between Maven and Gradle, Initializing the Project, Initializing the Project
- greenfield, Adding OpenID Connect and OAuth2 dependencies to Aircraft Positions-Adding OpenID Connect and OAuth2 dependencies to Aircraft Positions, Adding OpenID Connect and OAuth2 Dependencies to Aircraft Positions
- Groovy DSL, Gradle, Initializing the Project, Initializing the Project, End User Interactions Using Template Engines

H

- H2 database driver, Adding a Database Dependency, R2DBC with H2-Reactive Thymeleaf
- HEAD, Your API, HTTP Verb Style
- health indicator, Actuator, Actuator
- Heroku, Exploding JARs, Deploying Spring Boot Applications to Containers
- Hooks.onOperatorDebug(), Hooks.onOperatorDebug()-Checkpoints
- HTTP Firewall, The HTTP Firewall
- HTTP verbs, Your API, HTTP Verb Style-And More

  - DELETE, DELETE-ing, Trust, but Verify, Saving and Retrieving Data
  - GET, GET-ting-Let's GET Busy, Trust, but Verify, Saving and Retrieving Data
  - POST, POST-ing, Trust, but Verify, Saving and Retrieving Data
  - PUT, PUT-ting, Trust, but Verify, Saving and Retrieving Data
- HTTPie, Trust, but Verify, Saving and Retrieving Data-Saving and Retrieving Data, Actuator, Migrating Aircraft Positions to RSocket
- HTTPS, Broadcasting aircraft positions to WebSocket connections
- HttpSecurity, Refactoring PlaneFinder for resource authorization

I

- I/O (input/output), blocking versus non-blocking, Tomcat versus Netty, When Reactive?
- IDE (Integrated Development Environment), Staying In Integrated Development Environments (IDEs)-Staying In Integrated Development Environments (IDEs), Creating a Container Image from an IDE-Creating a Container Image from an IDE
- IETF (Internet Engineering Task Force), Your API, HTTP Verb Style, And More, What Is WebSocket?, Securing Your Spring Boot Application, Request and Response Headers
- imperative Java, Going Deeper with Reactive
- index.html file, Creating the requisite View files
- infix notation, Kotlin
- Initializr (see Spring Initializr)
- InMemoryUserDetailsManager, Adding Authentication
- inspection of the API (Actuator), Actuator-Turning Up the Volume on Logging with Actuator
- Integrated Development Environment (see IDE)
- IntelliJ IDEA, Staying In Integrated Development Environments (IDEs), @ConfigurationProperties
- IntelliJ IDEA Keybindings, Staying In Integrated Development Environments (IDEs)
- Internet Engineering Task Force (see IETF)
- IoC (Inversion of Control) container, @RestController in a Nutshell

J

- JAR files, Executable JARs for Simplified Deployment-Executable JARs for Simplified Deployment, Autoconfiguration Report, Deploying Your Spring Boot Application, Revisiting the Spring Boot Executable JAR-Building a "Fully Executable" Spring Boot JAR
- JarLauncher, Exploding JARs

- Java
  - choosing versus Kotlin, Java or Kotlin?, Choosing Between Java and Kotlin
  - executable JARs, Executable JARs for Simplified Deployment-Executable JARs for Simplified Deployment
  - Initializr, The Spring Initializr-The Spring Initializr
  - installing, The Spring Initializr-The Spring Initializr
  - language support for, Staying In Integrated Development Environments (IDEs)
  - versus Spring Boot application, Cruising Down main()-Cruising Down main()
- Java Virtual Machine (see JVM)
- JavaScript Object Notation (see JSON)
- JDK (Java Development Kit), The Spring Initializr-The Spring Initializr, Straight Outta Commandline, Revisiting the Spring Boot Executable JAR-Building a “Fully Executable” Spring Boot JAR
- JetBrains, Kotlin, Staying In Integrated Development Environments (IDEs)
- JPA (Java Persistence API)
  - adding database dependency, Adding a Database Dependency-Adding a Database Dependency
  - autoconfiguration report, Autoconfiguration Report
  - reactive data access, R2DBC with H2-Reactive Thymeleaf
  - RedisHash, Converting from Template to Repository
  - repository support, Repository Support, Creating a Repository-Based Service Using the Java Persistence API (JPA)-Other mechanisms
  - Spring MVC, Defining the domain class
- JpaRepository, The Repository
- JSON (JavaScript Object Notation)
  - authorization, Refactoring PlaneFinder for resource authorization
  - configuration, Turning Up the Volume on Logging with Actuator
  - GET, GET-ting, Let’s GET Busy, Saving and Retrieving Data
  - POST, POST-ing, Trust, but Verify, Saving and Retrieving Data
  - PUT, Trust, but Verify, Saving and Retrieving Data
  - Redis service, Developing the Redis Service-Converting from Template to Repository
  - RSocket, Migrating PlaneFinder to RSocket, Migrating Aircraft Positions to RSocket
  - testing reactive applications, But First, Refactoring
- JSON Web Token (see JWT)
- JVM (Java Virtual Machine)
  - executable JARs, Executable JARs for Simplified Deployment, Building a “Fully Executable” Spring Boot JAR
  - Java, Java or Kotlin?, Choosing Between Java and Kotlin

  - Kotlin, Kotlin-Choosing Between Java and Kotlin
  - main() method, Cruising Down main()-Cruising Down main()
  - Project Reactor, Project Reactor-Project Reactor
- JWT (JSON Web Token), Refactoring PlaneFinder for resource authorization-Refactoring PlaneFinder for resource authorization

K

- Kafka (see Apache Kafka)
- Kinesis (see Amazon Kinesis)
- Kotlin, Gradle, Kotlin-Choosing Between Java and Kotlin, Creating a Repository-Based Service Using a NoSQL Document Database-The results

L

- loadData(), A Bit of Polishing, Populating the database using the application's repository
- loadUserByUsername(), Adding Authentication
- logging levels (Actuator), Turning Up the Volume on Logging with Actuator-Turning Up the Volume on Logging with Actuator
- logging passwords, warning against, Adding Authentication
- Lombok
  - authorization, Refactoring Aircraft Positions for authentication and authorization-Refactoring Aircraft Positions for authentication and authorization
  - creating repository-based service, Polling PlaneFinder, Populating the database using the application's repository, Defining the domain class
  - entities, Defining Entities
  - project options, Initializing the Project
  - reactive data access, Updating PlaneFinder
  - Redis service development, Defining the domain class-Converting from Template to Repository
  - RSocket, Migrating Aircraft Positions to RSocket
  - template engines for end user interactions, Defining the domain class
- LTS (Long Term Support), Java, The Spring Initializr, Straight Outta Commandline

M

- main(), Cruising Down main()-Cruising Down main()
- MariaDB, Developing the JPA (MySQL) Service-Other mechanisms
- Maven (see Apache Maven)

- MFA (Multi-Factor Authentication), Authentication, Implementing OpenID Connect and OAuth2 for Authentication and Authorization
- Mockito, And Now, the Testing
- Model object, @RestController in a Nutshell
- Model-View-Controller (see Spring MVC)
- MongoDB, Creating a Repository-Based Service Using a NoSQL Document Database-The results, Updating the Aircraft Positions application-Updating the Aircraft Positions application
- MongoRepository, Creating the repository interface
- Mono, Project Reactor, Updating PlaneFinder, Testing Reactive Applications, But First, Refactoring
- Multi-Factor Authentication (see MFA)
- Mustache, End User Interactions Using Template Engines
- MySQL, Developing the JPA (MySQL) Service-Other mechanisms

N

- Neo4j, Creating a Repository-Based Service Using a NoSQL Graph Database-The results
- NetBeans (see Apache NetBeans)
- Netty, Tomcat versus Netty
- non-blocking backpressure, Reactive Streams, Introduction to Reactive Programming
- NoSQL document database, repository-based service for, Creating a Repository-Based Service Using a NoSQL Document Database-The results
- NoSQL graph database, repository-based service for, Creating a Repository-Based Service Using a NoSQL Graph Database-The results

O

- OAuth2, Implementing OpenID Connect and OAuth2 for Authentication and Authorization-Refactoring PlaneFinder for resource authorization
- OAuth2AuthorizedClientRepository, Refactoring Aircraft Positions for authentication and authorization
- OCI (Open Container Initiative), Deploying Spring Boot Applications to Containers
- Okta, Implementing OpenID Connect and OAuth2 for Authentication and Authorization, Adding OpenID Connect and OAuth2 dependencies to Aircraft Positions, Refactoring Aircraft Positions for authentication and authorization, Adding OpenID Connect and OAuth2 Dependencies to Aircraft Positions-Refactoring PlaneFinder for resource authorization

- onOperatorDebug(), Hooks.onOperatorDebug()
- OpenID Connect, Implementing OpenID Connect and OAuth2 for Authentication and Authorization-Refactoring PlaneFinder for resource authorization
- Operations interface, Template Support
- OPTIONS, Your API, HTTP Verb Style

P

- pack utility, Pack-Pack
- Paketo buildpacks, Deploying Spring Boot Applications to Containers-Dive
- passwords, Adding Authentication, Adding Authentication, Adding Authentication
- PATCH, Your API, HTTP Verb Style
- PlaneFinder, @Before
  - JPA service, Polling PlaneFinder-Other mechanisms
  - messages, Passing Messages-The results
  - polling, Bringing it all together-Converting from Template to Repository, Polling PlaneFinder, Polling PlaneFinder
  - reactive data access, R2DBC with H2-Reactive Thymeleaf
  - Redis service, Creating a Template-Based Service Using Redis-Converting from Template to Repository
- POJO (plain old Java object), @ConfigurationProperties, Potential Third-Party Option-Potential Third-Party Option
- polling PlaneFinder, Bringing it all together-Converting from Template to Repository, Polling PlaneFinder, Polling PlaneFinder
- pom.xml, Apache Maven
- POST, POST-ing, Trust, but Verify, Saving and Retrieving Data
- Project Reactor, Project Reactor-Reactive Thymeleaf
  - data access, Reactive Data Access
  - R2DBC with H2, R2DBC with H2-Reactive Thymeleaf
  - reactor-tools, ReactorDebugAgent.init()
  - RSocket, What Is RSocket?
  - Schedulers, Diagnosing and Debugging Reactive Applications
  - Tomcat versus Netty, Tomcat versus Netty
- Publisher, Project Reactor, Updating PlaneFinder, Updating PlaneFinder, Reactive Thymeleaf, Testing Reactive Applications, But First, Refactoring, Test for retrieving all aircraft positions, Hooks.onOperatorDebug(), Checkpoints-Checkpoints
- PUT, PUT-ting, Trust, but Verify, Saving and Retrieving Data

Q

- QueryByExampleExecutor, Creating the repository interface

R

- R2DBC (Reactive Relational Database Connectivity), R2DBC with H2-Reactive Thymeleaf
- RabbitMQ, Passing Messages, Required dependencies-The results, Broadcasting aircraft positions to WebSocket connections
- reactive programming, Reactive Programming with Project Reactor and Spring WebFlux-Migrating Aircraft Positions to RSocket, Going Deeper with Reactive-ReactorDebugAgent.init()
  - diagnosing and debugging, Diagnosing and Debugging Reactive Applications-ReactorDebugAgent.init()
  - Project Reactor, Project Reactor-Reactive Thymeleaf
  - RSocket, RSocket for Fully Reactive Interprocess Communication-Migrating Aircraft Positions to RSocket
  - testing, Testing Reactive Applications-Testing Aircraft Positions search capabilities
  - Thymeleaf support for RS, Reactive Thymeleaf
- Reactive Streams (RS), Introduction to Reactive Programming-Reactive Thymeleaf
- ReactiveCrudRepository, Updating PlaneFinder, Updating the Aircraft Positions application
- Reactor (see Project Reactor)
- reactor-tools, ReactorDebugAgent.init()
- ReactorDebugAgent.init(), ReactorDebugAgent.init()-ReactorDebugAgent.init()
- Redis database service, Creating a Template-Based Service Using Redis-The results, Converting from Template to Repository, Defining the domain class
- RedisAccessor, Adding template support
- RedisConnectionFactory, Adding template support, Bringing it all together
- RedisHash, Converting from Template to Repository, Defining the domain class
- RedisOperations, Adding template support, Bringing it all together, Converting from Template to Repository
- RedisTemplate, Adding template support, Bringing it all together
- repository, Repository Support
  - connecting to database, The Repository-The utility, aka “Springing” into action
  - converting from template to, Converting from Template to Repository-Converting from Template to Repository

    - JPA and, Repository Support, Creating a Repository-Based Service Using the Java Persistence API (JPA)-Other mechanisms
    - NoSQL document database, Creating a Repository-Based Service Using a NoSQL Document Database-The results
    - NoSQL graph database, Creating a Repository-Based Service Using a NoSQL Graph Database-The results
- request-response model, RSocket for Fully Reactive Interprocess Communication, Request and Response Headers
- request-stream model, Migrating Aircraft Positions to RSocket
- resource authorization, PlaneFinder Resource Server-Refactoring PlaneFinder for resource authorization
- response headers, Adding Spring Security Dependencies-Adding Spring Security Dependencies
- REST API, Creating Your First Spring Boot REST API-Trust, but Verify
    - creating a domain, Creating a Simple Domain-Creating a Simple Domain
    - HTTP verbs, Your API, HTTP Verb Style-And More
    - Initializr, Back to the Initializr-Back to the Initializr
    - inspection of (Actuator), Actuator-Turning Up the Volume on Logging with Actuator
    - testing the code, Trust, but Verify-Trust, but Verify
- RestApiDemoController, Let's GET Busy-PUT-ting, The utility, aka "Springing" into action-The utility, aka "Springing" into action, A Bit of Polishing
- RocketMQ (see Apache RocketMQ)
- RS (see Reactive Streams)
- RSocket, RSocket for Fully Reactive Interprocess Communication-Migrating Aircraft Positions to RSocket, But First, Refactoring
- RSocketRequester, Migrating Aircraft Positions to RSocket, But First, Refactoring-But First, Refactoring, And Now, the Testing
- rwx (read, write, execute), Building a "Fully Executable" Spring Boot JAR

S

- SburRedisApplication, Converting from Template to Repository
- Schedulers, Diagnosing and Debugging Reactive Applications
- SCSt (Spring Cloud Stream), Autoconfiguration, Passing Messages-Required dependencies
- SDKMAN!, The Spring Initializr, Straight Outta Commandline
- security (see Spring Security)
- SecurityConfig, Adding Authentication, Authorization-Authorization, Refactoring Aircraft Positions for authentication and authorization, Refactoring PlaneFinder

for resource authorization
- SecurityWebFilterChain, Refactoring PlaneFinder for resource authorization
- securityWebFilterChain(), Refactoring PlaneFinder for resource authorization
- ServerHttpSecurity, Refactoring PlaneFinder for resource authorization
- setUp(), And Now, the Testing
- slices (image layers), Creating a Container Image from an IDE
- Solace PubSub+, Supplying aircraft positions
- SpEL (Spring Expression Language), @Value, @Value, Becoming More Environmentally Aware Using Actuator
- Spring Authorization Server, Implementing OpenID Connect and OAuth2 for Authentication and Authorization
- Spring Boot
  - features, Spring Boot's Three Foundational Features-Autoconfiguration
  - IDEs, Staying In Integrated Development Environments (IDEs)-Staying In Integrated Development Environments (IDEs)
  - installing the CLI, Straight Outta Commandline-Straight Outta Commandline
  - versus standard Java application, Cruising Down main()-Cruising Down main()
  - tools for, Choosing Your Tools and Getting Started-Cruising Down main()
  - version to use, Choosing a Version of Spring Boot
- Spring Boot Actuator, Actuator-Turning Up the Volume on Logging with Actuator
  - environmental awareness, Becoming More Environmentally Aware Using Actuator-Becoming More Environmentally Aware Using Actuator
  - logging levels, Turning Up the Volume on Logging with Actuator-Turning Up the Volume on Logging with Actuator
  - opening up, Getting Actuator to Open Up
- Spring Boot CLI, Straight Outta Commandline-Straight Outta Commandline
- Spring Boot Extension Pack, Staying In Integrated Development Environments (IDEs)
- Spring Boot Gradle plug-in, Revisiting the Spring Boot Executable JAR, Deploying Spring Boot Applications to Containers
- Spring Boot Maven plug-in, Revisiting the Spring Boot Executable JAR-Building a "Fully Executable" Spring Boot JAR, Deploying Spring Boot Applications to Containers
- Spring Boot Starter library, Initializing the Project, Adding OpenID Connect and OAuth2 dependencies to Aircraft Positions
- Spring Data, Really Digging into Data-The results, But First, Refactoring
  - (see also database access)
  - developing JPA (MySQL) service, Creating a Repository-Based Service Using the Java Persistence API (JPA)-The results
  - loading data, Loading Data-Other mechanisms

    - NoSQL document database, Creating a Repository-Based Service Using a NoSQL Document Database-The results
    - NoSQL graph database, Creating a Repository-Based Service Using a NoSQL Graph Database-The results
    - query derivation, But First, Refactoring
    - Redis for template-based service creation, Creating a Template-Based Service Using Redis-The results
    - template conversion to repository, Converting from Template to Repository-Converting from Template to Repository
- Spring Environment, Application Configuration
- Spring Expression Language (see SpEL)
- Spring Framework, Kotlin
- Spring Initializr (start.spring.io), The Spring Initializr-The Spring Initializr, Back to the Initializr-Back to the Initializr, Required dependencies
- Spring MVC (Model-View-Controller), Creating Applications Using Spring MVC-The results
    - end user interactions using template engines, End User Interactions Using Template Engines-A refreshing flourish
    - messages, Passing Messages-The results
    - purpose, @RestController in a Nutshell
    - versus Spring WebFlux, Tomcat versus Netty
    - WebSocket, Creating Conversations with WebSocket-The results
- Spring Security, Actuator, Actuator, Securing Your Spring Boot Application-Refactoring PlaneFinder for resource authorization
    - (see also authentication and authorization)
    - filter chains, Security Filter Chains
    - forms-based authentication, Adding Authentication-Adding Authentication
    - forms-based authorization, Authorization-Authorization
    - HTTP Firewall, The HTTP Firewall
    - OpenID Connect and OAuth2, Implementing OpenID Connect and OAuth2 for Authentication and Authorization-Refactoring PlaneFinder for resource authorization, Adding OpenID Connect and OAuth2 Dependencies to Aircraft Positions
    - request-response model, Request and Response Headers
- Spring Web, Initializing the Project
- Spring WebFlux, Tomcat versus Netty, Reactive Thymeleaf, Authorization, When Reactive?
- SQL (see MySQL)
- stacktrace, Diagnosing and Debugging Reactive Applications-Diagnosing and Debugging Reactive Applications, Hooks.onOperatorDebug()

- startup, application, Cruising Down main()-Cruising Down main()
- stateless applications/services, What Is REST, and Why Does It Matter?
- StepVerifier, Test for retrieving all aircraft positions, Testing Aircraft Positions search capabilities
- StringRedisSerializer, Adding template support
- Sublime Text, Staying In Integrated Development Environments (IDEs)
- Subscriber, Diagnosing and Debugging Reactive Applications

T

- TCK (Technology Compatibility Kit), Introduction to Reactive Programming
- TDD (Test Driven Development), The utility, aka “Springing” into action, Testing Reactive Applications
- templates and template engines
  - conversion to repository, Converting from Template to Repository-Converting from Template to Repository
  - end user interactions using, End User Interactions Using Template Engines-A refreshing flourish
  - Redis service, Creating a Template-Based Service Using Redis-The results
  - Spring Data support, Template Support
- testing, Testing Reactive Applications-Testing Aircraft Positions search capabilities
  - first REST API code, Trust, but Verify-Trust, but Verify
  - refactoring, But First, Refactoring-But First, Refactoring
  - Spring Security, Adding Spring Security Dependencies
- Thymeleaf, End User Interactions Using Template Engines, Working with Model and Controller, Creating the requisite View files-A refreshing flourish, Updating the Aircraft Positions application-Reactive Thymeleaf
- Tomcat, Tomcat versus Netty

U

- UserDetailsManager, Adding Authentication
- UserDetailsPasswordService, Adding Authentication
- UserDetailsService, Adding Authentication

V

- view technologies (see Spring MVC)
- ViewResolver, @RestController in a Nutshell

- VSCode (Visual Studio Code), Staying In Integrated Development Environments (IDEs)
- VSCodium, Staying In Integrated Development Environments (IDEs)

W

- W3C (World Wide Web Consortium), Securing Your Spring Boot Application, Request and Response Headers
- WAR file, Deploying Your Spring Boot Application
- WebClient, Initializing the Project, Bringing it all together, Polling PlaneFinder, Polling PlaneFinder, Polling PlaneFinder, Refactoring Aircraft Positions for authentication and authorization-Refactoring Aircraft Positions for authentication and authorization, When Reactive?
- WebFlux (see Spring WebFlux)
- WebSecurityConfigurerAdapter, Authorization, Refactoring Aircraft Positions for authentication and authorization
- WebSocket Secure, Broadcasting aircraft positions to WebSocket connections
- WebSocketConfigurer, Broadcasting aircraft positions to WebSocket connections
- WebSocketHandler, Handling WebSocket connections and messages
- WebTestClient, And Now, the Testing
- World Wide Web Consortium (see W3C)

X

- X-Content-Type-Options response header, Adding Spring Security Dependencies
- X-Frame-Options response header, Adding Spring Security Dependencies
- X-XSS-Protection response header, Adding Spring Security Dependencies
- XSS (Cross Site Scripting), Adding Spring Security Dependencies

Previous chapter

< 12. Going Deeper with Reactive

Next chapter

About the Author >